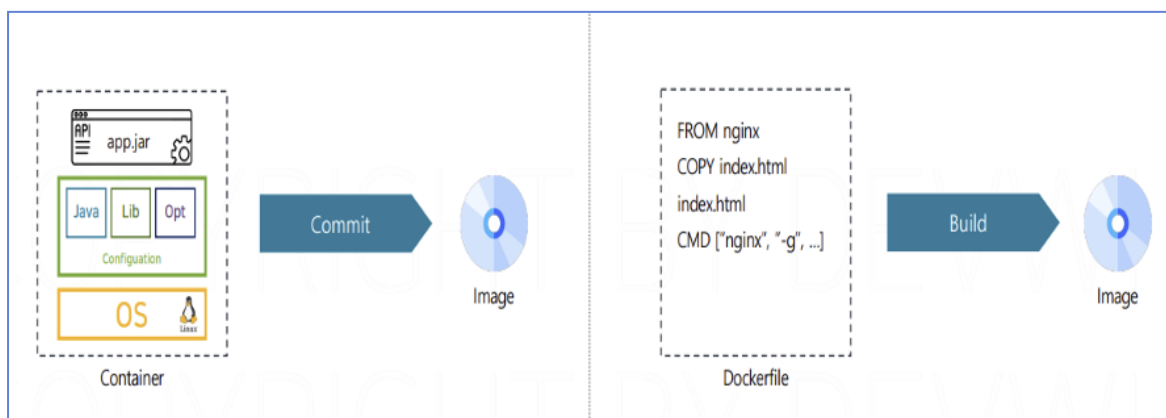


컨테이너로 이미지 만들기

나만의 이미지로 동일한 구성의 컨테이너를 대량으로 생성 할 수 있고 다른 서버로 쉽게 이동할 수도 있다.

이미지 만드는 방법

- 1) 커밋 : 컨테이너의 현재 상태를 이미지로 저장
 - 2) 빌드 : Dockerfile(도커 이미지 생성관련 스크립트 파일)을 통해 이미지를 생성하여 저장
- Dockerfile** 이란 **Docker** 이미지를 만들게 해주는 파일



- 1) commit 명령으로 컨테이너를 이미지로 변환하기

docker commit -m 커밋명 실행중인컨테이너명 생성할이미지명
 실행 중인 컨테이너를 이미지로 생성

[도커 허브의 개인레지스트리명(계정명)/]이미지이름

- 2) Dockerfile 스크립트로 이미지 생성하기

docker build -t 이미지명 Dockerfile경로
 도커파일을 통해 이미지 빌드

docker build -f 도커파일명 -t 이미지명 Dockerfile경로
 도커파일명이 Dockerfile이 아닌 경우 별도 지정

```
FROM nginx
WORKDIR /usr/share/nginx/html
RUN mkdir images
COPY docker.png images/docker.png
COPY my.html .
EXPOSE 80
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

FROM 이미지명
베이스 이미지를 지정

COPY 파일경로 복사할경로
파일을 레이어에 복사

CMD ["명령어"]
컨테이너 실행 시 명령어 지정

Dockerfile 스크립트에 사용되는 주요 명령

주요 Dockerfile 인스트럭션

인스트럭션	내용
FROM	토대가 되는 이미지를 지정
ADD	이미지에 파일이나 폴더를 추가
COPY	이미지에 파일이나 폴더를 추가
RUN	이미지를 빌드할 때 실행할 명령어를 지정
CMD	컨테이너를 실행할 때 실행할 명령어를 지정
ENTRYPOINT	컨테이너를 실행할 때 실행할 명령어를 강제 지정
ONBUILD	이 이미지를 기반으로 다른 이미지를 빌드할 때 실행할 명령어를 지정
EXPOSE	이미지가 통신에 사용할 포트를 명시적으로 지정
VOLUME	퍼시스턴스 데이터를 저장할 경로를 명시적으로 지정
ENV	환경변수를 정의
WORKDIR	RUN, CMD, ENTRYPOINT, ADD, COPY에 정의된 명령어를 실행하는 작업 디렉터리를 지정
SHELL	빌드 시 사용할 셸을 변경
LABEL	이름이나 버전, 저자 정보 설정
USER	RUN, CMD, ENTRYPOINT에 정의된 명령어를 실행하는 사용자 또는 그룹을 지정
ARG	docker build 커맨드를 사용할 때 입력받을 수 있는 인자를 선언
STOPSIGNAL	docker stop 커맨드를 사용할 때 컨테이너 안에서 실행 중인 프로그램에 전달되는 시그널을 변경
HEALTHCHECK	컨테이너 헬스체크 방법을 커스터마이징

[Dockerfile의 작성 방법]

Docker가 대중화되면서 많은 프로젝트들이 개발 환경을 컨테이너화(containerization)시키고 있다. 이러한 프로젝트의 최상위 디렉터리에서는 Dockerfile이 위치하게 되며, Dockerfile의 작성방법을 이해하는 것은 그 프로젝트의 개발 환경이 어떻게 구성되는지 이해하는 첫걸음이다.

Dockerfile 은 Docker 이미지(image)가 어떤 단계를 거쳐 빌드(build)되어야 하는지를 담고 있는 스크립트 파일이다. Docker 는 Dockerfile 에 나열된 명령문을 차례대로 수행해서 이미지를 생성한다.

다음은 자주 쓰이는 명령어 위주로 Dockerfile 을 작성하는 방법에 대해서 정리한 내용이다.

명령어(INSTRUCTION) 인자(arguments)

각 명령문은 명령어로 시작하고 여러 개의 인자가 정의될 수 있으며, 해당 명령문에 대한 주석(#)도 정의할 수 있다. 인자와 구분이 쉽도록 명령어는 모두 영문 대문자로 작성하는 것이 관례이다.

FROM 명령문

FROM <이미지>

FROM <이미지>:<태그>

하나의 Docker 이미지는 base 이미지부터 시작해서 기존 이미지위에 새로운 이미지를 중첩해서 여러 단계의 이미지 층(layer)을 쌓아가며 만들어진다.

FROM 명령문은 이 base 이미지를 지정하기 위해서 사용되는데, 보통 Dockerfile 내에서 최상단에 위치한다. base 이미지는 일반적으로 Docker Hub 와 같은 Docker repository 에 올려놓은 잘 알려진 공개 이미지인 경우가 많다.

예) Ubuntu 최신 버전을 base 이미지로 사용

```
FROM ubuntu:latest
```

예) NodeJS 12 를 base 이미지로 사용

```
FROM node:12
```

예) Python 3.8 (alpine 리눅스 기반)을 base 이미지로 사용

```
FROM python:3.8-alpine
```

예) 아파치 웹 서버를 base 이미지로 사용

```
FROM httpd
```

예) 최신 버전의 MySQL 을 base 이미지로 사용

```
FROM mysql:latest
```

예) openjdk 17 버전을 base 이미지로 사용

```
FROM openjdk:17-jdk
```

WORKDIR 명령문

WORKDIR 명령문은 셸(shell)의 cd 명령문처럼 컨테이너 상에서 작업디렉토리로 전환을 위해서 사용된다. WORKDIR 명령문으로 작업 디렉토리를 전환하면 그 이후에 등장하는 모든 RUN, CMD, ENTRYPOINT, COPY, ADD 명령문은 해당 디렉토리를 기준으로 실행된다. 즉, WORKDIR 은 도커 이미지의 어디에서 작업을 할지를 의미한다.

WORKDIR <이동할 경로>

예) WORKDIR /usr/app #/usr/app 으로 작업 디렉토리 전환

RUN 명령문

RUN [<커맨드>, "<파라미터 1>", "<파라미터 2>"]

RUN <전체 커맨드>

RUN 명령문은 마치 셸(shell)에서 커맨드를 실행하는 것처럼 이미지 빌드 과정에서 필요한 커맨드를 실행하기 위해서 사용된다. 셸(shell)을 통해 거의 못하는 작업이 없는 것처럼 RUN 명령문으로 할 수 있는 작업은 제한 없지만 보통 이미지 안에 특정 소프트웨어를 설치하기 위해서 많이 사용된다.

curl 도구 설치

예) RUN apk add curl

npm 패키지 설치

예) RUN npm install --silent

pip 패키지 설치

예) RUN pip install -r requirements.txt

ENTRYPOINT 명령문

ENTRYPOINT [<커맨드>, "<파라미터 1>", "<파라미터 2>"]

ENTRYPOINT <전체 커맨드>

ENTRYPOINT 명령문은 이미지를 컨테이너로 띄울 때 항상 실행되어야 하는 커맨드를 지정할 때 사용한다. ENTRYPOINT 명령문은 Docker 이미지를 마치 하나의 실행 파일처럼 사용할 때 유용하다. 컨테이너가 기동 될 때 ENTRYPOINT 명령문으로 지정된 커맨드가 실행되고, 이 커맨드로 실행된 프로세스가 죽을 때, 컨테이너도 따라서 종료되기 때문이다.

npm start 스크립트 실행

예) ENTRYPOINT ["npm", "start"]

nginx 서버 기동

예) ENTRYPOINT ["nginx", "-g", "daemon off;"]

nginx : nginx 웹 서버를 실행하라는 명령어

-g : nginx 설정을 명령줄에서 직접 지정할 때 사용하는 옵션이다.

"daemon off;" : nginx 가 백그라운드(daemon) 모드로 실행되지 않도록 설정해주는 명령어이다.

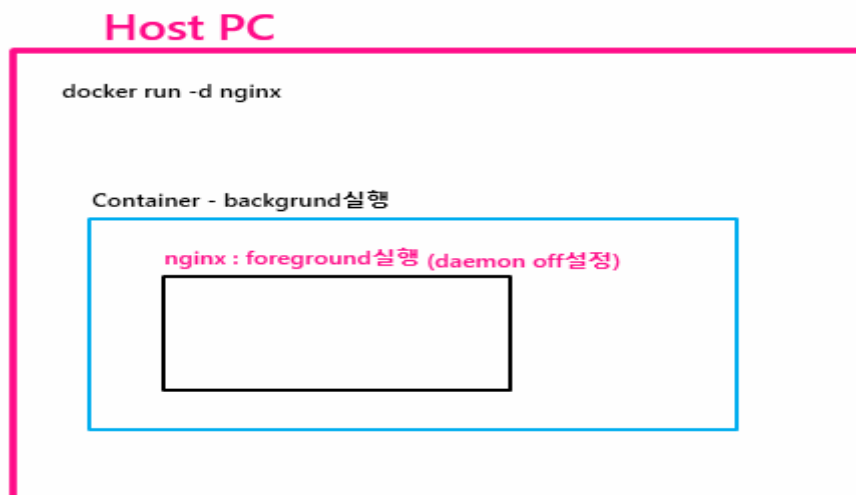
왜 daemon off;가 필요한가?

Docker 컨테이너는 기본적으로 **포그라운드에서 실행 중인 프로세스가 있어야** 컨테이너가 살아 있다. 그런데 Nginx 는 기본적으로 **백그라운드(daemon)로 실행되기 때문에**, 그냥 실행하면 **컨테이너가 바로 종료돼 버린다**.

그래서 daemon off; 옵션을 줘서 nginx 가 **포그라운드에서 실행되게** 만들면, Docker 는 그걸 "컨테이너가 잘 실행되고 있다"고 판단하고 컨테이너가 종료되지 않는다.

docker 의 -d 는 컨테이너 자체가 백그라운드로 실행되는 것

반대로 nginx 가 daemon on 이면 컨테이너 안에서 Nginx 가 백그라운드로 실행됨



CMD 명령문

CMD ["<커맨드>", "<파라미터 1>", "<파라미터 2>"]

CMD ["<파라미터 1>","<파라미터 2>"]

CMD <전체 커맨드>

CMD 명령문은 해당 이미지를 컨테이너로 띄울 때 디폴트로 실행할 커맨드나, ENTRYPOINT 명령문으로 지정된 커맨드에 디폴트로 넘길 파라미터를 지정할 때 사용한다.

CMD 명령문은 많은 경우, ENTRYPOINT 명령문과 함께 사용하게 되는데, ENTRYPOINT 명령문으로는 커맨드를 지정하고, CMD 명령문으로 디폴트 파라미터를 지정해주면 매우 유연하게 이미지를 실행할 수 있게 된다.

예를 들어, node 커맨드로 디폴트로 index.js 를 실행하되, docker run 커맨드에 인자가 있는 경우, 해당 인자를 실행하고 싶은 경우, 다음과 같이 Dockerfile 을 작성한다.

ENTRYPOINT ["node"]

CMD ["index.js"]

그러면 다음과 같이 docker run 커맨드의 인자 유무에 따라 node 커맨드로 다른 파일이 실행되게 할 수 있다.

node index.js 실행

\$ docker run test

node main.js 실행

\$ docker run test main.js

CMD 명령문과 RUN 명령문이 기능이 비슷해 보이는데 RUN 명령문은 이미지 빌드시 항상 실행되며, 하나의 Dockerfile 에 여러 개의 RUN 명령문을 선언할 수 있다. 반면에, CMD 명령문은 이미지를 컨테이너로 기동시킬 때 한 번의 실행 기회를 가지게 된다.

EXPOSE 명령문

EXPOSE <포트>

EXPOSE <포트>/<프로토콜>

EXPOSE 명령문은 네트워크 상에서 컨테이너로 들어오는 트래픽(traffic)을 리스닝(listening)하는 포트와 프로토콜을 지정하기 위해서 사용된다. 프로토콜은 TCP 와 UDP 중 선택할 수 있는데 지정하지 않으면 TCP 가 기본값으로 사용된다.

여기서 주의할 점은 EXPOSE 명령문으로 지정된 포트는 해당 컨테이너의 내부에서만 유효하며, 호스트(host) 컴퓨터에서는 이 포트를 바로 접근을 할 수 없다. 호스트 컴퓨터로부터 해당

포트로의 접근을 허용하려면, `docker run` 커맨드에서 `-p` 옵션을 통해 호스트 컴퓨터의 특정 포트를 포워딩(forwarding)시켜주어야 한다.

80/TCP 포트로 리스닝

예) `EXPOSE 80`

9999/UDP 포트로 리스닝

예) `EXPOSE 9999/udp`

COPY/ADD 명령문

`COPY <src>... <dest>`

`COPY ["<src>"... "<dest>"]`

`COPY` 명령문은 호스트 컴퓨터에 있는 디렉터리나 파일을 Docker 이미지의 파일 시스템으로 복사하기 위해서 사용된다. 절대 경로와 상대 경로를 모두 지원하며, 상대 경로를 사용할 때는 이전에 등장하는 `WORKDIR` 명령문으로 작업 디렉터리를 어디로 변경했는지 고려해야 한다.

package.json 파일만 복사

예) `COPY package.json package.json`

이미지 빌드를 수행한 디렉터리의 모든 파일을 컨테이너의 `app/` 디렉터리로 복사

예)

`WORKDIR app/`

`COPY . .`

`ADD` 명령문은 좀 더 파워풀한 `COPY` 명령문이라고 할 수 있다. `ADD` 명령문은 일반 파일뿐만 아니라 압축 파일이나 네트워크 상의 파일도 사용할 수 있다. 이렇게 특수한 파일을 다루는 게 아니라면 `COPY` 명령문을 사용하는 것이 권장된다.

ENV 명령문

`ENV <키><값>`

`ENV <키>=<값>`

`ENV` 명령문은 환경 변수를 설정하기 위해서 사용한다. `ENV` 명령문으로 설정된 환경 변수는 이미지 빌드 시에도 사용되고, 해당 컨테이너 안에서 실행되는 애플리케이션에서도 접근할 수 있다.

NODE_ENV 환경 변수를 production 으로 설정

예) ENV NODE_ENV production

자바 버전을 11 로 설정

예) ENV JAVA_VERSION 11

ARG 명령문

ARG <이름>

ARG <이름>=<기본값>

ARG 명령문은 docker build 커맨드로 이미지 빌드 시, --build-arg 옵션을 통해 넘길 수 있는 인자를 정의하기 위해 사용한다.

예를 들어, Dockerfile 에 다음과 같이 ARG 명령문으로 port 를 인자로 선언해주면,

ARG port

다음과 같이 docker build 커맨드에 --build-arg 옵션에 port 값을 넘길 수가 있다.

\$ docker build --build-arg port=8080.

인자의 디폴트값을 지정해주면, --build-arg 옵션으로 해당 인자가 넘어오지 않았을 때 사용된다.

ARG port=8080

설정된 인자 값은 다음과 같이 \${인자명} 형태로 읽어서 사용할 수 있다.

CMD start.sh -h 127.0.0.1 -p \${port}

ENV 와 달리 ARG 로 설정한 값은 이미지가 빌드되는 동안에만 유효하다.

.dockerignore 파일

명령문은 아니지만 .dockerignore 파일도 알아두면 Dockerfile 을 작성할 때 유용하다. Docker 이미지를 빌드할 때 제외 시키고 싶은 파일이 있다면, .dockerignore 파일에 추가한다.

예를 들어, .git 디렉터리와 마크다운(markdown) 파일을 모두 제외 시키고 싶다면 다음과 같이 .dockerignore 파일을 작성해주면 된다.

.dockerignore 파일

.git

*.md

이렇게 설정을 해주면 Docker 는 프로젝트 최상위 디렉터리에 위치하고 있는 markdown 파일들을 무시하게 되므로, RUN 과 CMD, COPY 와 같은 명령문이 해당 파일을 사용할 수 없게 된다.

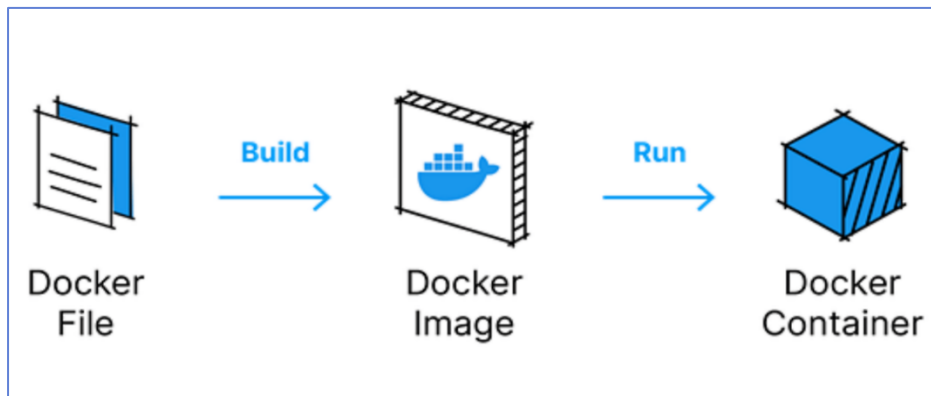
FROM	base 이미지 설정
WORKDIR	작업 디렉터리 설정
RUN	이미지 빌드 시 커맨드 실행
ENTRYPOINT	이미지를 가지고 컨테이너를 기동할 때 실행되어야 하는 커맨드 설정
CMD	ENTRYPOINT 에 지정된 명령 실행시 전달될 파라미터 설정
EXPOSE	컨테이너가 리스닝할 포트 및 프로토콜 설정
COPY/ADD	이미지의 파일 시스템으로 파일 또는 디렉터리 복사
ENV	환경 변수 설정
ARG	빌드 시 전달될 수 있는 인자 설정

(*) Docker 파일 작성시 참고할 사이트

<https://docs.docker.com/reference/>

<https://github.com/komljen/dockerfile-examples>

<https://github.com/topics/dockerfile-examples?o=desc&s=updated>



자주쓰는 명령어 요약 정리

FROM: 베이스 이미지 생성

FROM 은 베이스 이미지를 생성하는 역할을 한다. Docker 컨테이너를 **특정 초기 이미지**를 기반으로 추가적인 셋팅을 할 수 있다. 여기서 얘기한 '특정 초기 이미지'가 곧 **베이스 이미지**이다.

```
# 문법
FROM [이미지명]
FROM [이미지명]:[태그명]
```

#종료된 컨테이너에 들어가서 디버깅하고 싶을 때

이미지를 만들면서 컨테이너를 실행시켜보면, 컨테이너의 특성상 명령어 처리가 다 끝나는 대로 컨테이너가 종료된다. 그러다 보니 내부적으로 어떻게 컨테이너가 형성됐는지 디버깅을 하는데 어려움을 겪는다. 아래의 명령어를 추가함으로써 컨테이너가 바로 종료되는 걸 막을 수 있다. 그런 뒤에 docker exec -it 를 활용해 컨테이너 내부에 직접 들어가서 디버깅을 하면 된다.

```
# 500 초 동안 시스템을 일시정지 시키는 명령어
ENTRYPOINT ["/bin/bash", "-c", "sleep 500"]
```

COPY : 파일 복사(이동)

COPY 는 **호스트 컴퓨터**에 있는 파일을 복사해서 **컨테이너**로 전달한다.

```
# 문법
COPY [호스트 컴퓨터에 있는 복사할 파일의 경로] [컨테이너에서 파일이 위치할 경로]

# 예시
COPY app.txt /app.txt
```

.dockerignore 사용해보기

특정 파일 또는 폴더만 COPY 를 하고 싶지 않을 수 있다. 그럴 때 .dockerignore 를 활용한다.

```
.dockerignore
```

```
readme.txt
```

ENTRYPOINT : 컨테이너가 시작할 때 실행되는 명령어

ENTRYPOINT 는 컨테이너가 생성되고 최초로 실행할 때 수행되는 명령어를 뜻한다. 쉽게 설명하자면 ENTRYPOINT 에는 미니 컴퓨터의 전원을 켜고나서 실행시키고 싶은 명령어를 적으면

된다.

```
# 문법
ENTRYPOINT [명령문...]
```

```
# 예시
ENTRYPOINT ["node", "dist/main.js"]
```

```
FROM ubuntu
```

```
ENTRYPOINT ["/bin/bash", "-c", "echo hello"]
```

#RUN

RUN 은 이미지 생성 과정에서 명령어를 실행시켜야 할 때 사용한다.

```
# 문법
RUN [명령문]
```

```
# 예시
RUN npm install
```

RUN vs ENTRYPOINT

RUN 명령어와 ENTRYPOINT 명령어가 헷갈릴 때가 있다. 둘 다 같이 명령어를 실행시키기 때문이다. 하지만 엄연히 둘의 사용 용도는 다르다. RUN 은 '이미지 생성 과정'에서 필요한 명령어를 실행시킬 때 사용하고, ENTRYPOINT 는 생성된 이미지를 기반으로 컨테이너를 생성한 직후에 명령어를 실행시킬 때 사용한다.

```
FROM ubuntu
```

```
RUN apt update && apt install -y git
```

```
ENTRYPOINT ["/bin/bash", "-c", "sleep 500"]
```

WORKDIR : 작업 디렉토리를 지정

WORKDIR 으로 작업 디렉토리를 전환하면 그 이후에 등장하는 모든 RUN, CMD, ENTRYPOINT, COPY, ADD 명령문은 해당 디렉토리를 기준으로 실행된다. 작업 디렉토리를 굳이 지정해주는

이유는 컨테이너 내부의 폴더를 깔끔하게 관리하기 위해서이다. 컨테이너도 미니 컴퓨터와 같기 때문에 Dockerfile 을 통해 생성되는 파일들을 특정 폴더에 정리해두는 것이 추후에 관리가 쉽다. 만약 WORKDIR 을 쓰지 않으면 컨테이너 내부에 존재하는 기존 파일들과 뒤섞여버린다.

```
# 문법
WORKDIR [작업 디렉토리로 사용할 절대 경로]

# 예시
WORKDIR /usr/src/app
```

예시)

<pre>FROM ubuntu WORKDIR /my-dir → 컨테이너 안에서 작업할 작업디렉토리를 /my-dir 설정(이후 나오는 COPY, RUN 는 모두 이 디렉토리 안에서 실행됨 (cd /my-dir 과 같은 효과) COPY ./ ./ → 호스트 pc 현재폴더의 모든 파일을 컨테이너의 작업폴더(/my-dir)안으로 복사 ENTRYPOINT ["/bin/bash", "-c", "sleep 500"] → 컨테이너 시작되면 명령어 실행한다. 컨테이너가 500초동안 sleep(테스트나 디버깅하기 위해서 컨테이너 바로 종료되지 않도록 설정</pre>	
--	--

#EXPOSE : 컨테이너 내부에서 사용 중인 포트를 문서화하기

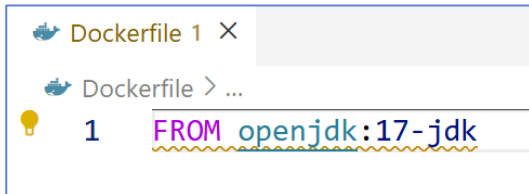
EXPOSE 는 컨테이너 내부에서 어떤 포트에 프로그램이 실행되는 지를 문서화하는 역할만 한다. docker -p 8080:8080 ... 와 같은 명령어의 -p 옵션과 같은 역할은 하지 않는다. 쉽게 표현하자면 EXPOSE 명령어는 쓰나 안 쓰나 작동하는 방식에는 영향을 미치지 않는다.

```
# 문법
EXPOSE [포트 번호]

# 예시
EXPOSE 3000
```

실습하기 - Dockerfile 명령어 TEST

1) FROM 명령어 - jdk17 이미지 만들기



#이미지 만들기 명령어

docker build -t 이미지이름 .

만든 이미지 실행하기

docker run -d 이미지이름

컨테이너 내부 bin/bash 로 들어가기

docker exec -it 컨테이너이름 /bin/bash

C:\Users\W\KOSTA>docker exec -it f192 /bin/bash
Error response from daemon: container f192d535b281042bc84f5e4ef7bc1846a808e5655c57fb62943cf47f727cd4fd is not running

☞ 컨테이너 내부 bash 셸로 들어갈 수 없다. 컨테이너가 기동 될 때 ENTRYPOINT 명령문으로 지정된 커맨드가 실행되고, 이 커맨드로 실행된 프로세스가 죽을 때, 컨테이너도 따라서 종료된다.

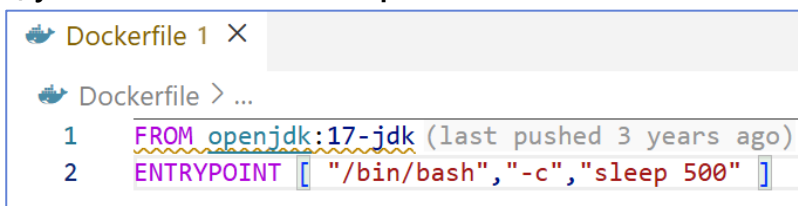
#종료된 컨테이너에 들어가서 디버깅하고 싶을 때

이미지를 만들면서 컨테이너를 실행시켜보면, 컨테이너의 특성상 명령어 처리가 다 끝나는 대로 컨테이너가 종료된다. 그러다 보니 내부적으로 어떻게 컨테이너가 형성됐는 지 디버깅을 하는데 어려움을 겪는다. 아래의 명령어를 추가함으로써 컨테이너가 바로 종료되는 걸 막을 수 있다. 그런 뒤에 docker exec -it 를 활용해 컨테이너 내부에 직접 들어가서 디버깅을 하면 된다.

500 초 동안 시스템을 일시정지 시키는 명령어

ENTRYPOINT ["/bin/bash", "-c", "sleep 500"]

2) jdk17 이미지 만들기 - sleep 를 설정해서 500 초동안 실행대기



3) **COPY** 명령어 - ubuntu 운영체제로 host PC 파일 복사하기

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2 COPY index.html /app.html
3 ENTRYPOINT [ "/bin/bash", "-c", "sleep 500" ]

```

4) **COPY** 명령어

ubuntu 운영체제로 host PC 파일, 폴더 복사하기 (Docker 쪽 폴더명 끝에 / 필수)

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2 COPY test /test/
3 COPY index.html /index.html
4 ENTRYPOINT [ "/bin/bash", "-c", "sleep 500" ]

```

5) **COPY** 명령어 - ubuntu 운영체제로 host PC 모든 txt 파일 특정 폴더로 복사하기

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2 COPY *.txt /text-file/
3
4 ENTRYPOINT [ "/bin/bash", "-c", "sleep 500" ]

```

6) **.dockerignore** 파일 (.dockerignore 를 이용하여 특정파일 제외시키기)

```

Dockerfile X .dockerignore
Dockerfile > ...
1 FROM ubuntu
2 COPY ./ /
3
4 ENTRYPOINT [ "/bin/bash", "-c", "sleep 500" ]

```

```

.dockerignore X
.dockerignore
1 # 모든 하위 디렉터리를 포함해서 .html 확장자를 가진
2 # 모든 파일을 무시(ignore) 하겠다는 의미
3 **/*.html

```

7) **ENTRYPOINT** 명령어

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2
3 # ENTRYPOINT는 컨테이너가 생성되고 최초로 실행할때 수행되는 명령어
4 ENTRYPOINT [ "/bin/bash", "-c", "echo hello" ]

```

docker logs 컨테이너아이디로 확인하기

8) **RUN** 명령어 ex)우분투 생성하는 과정에 git 을 설치하고 싶다.

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2
3 # RUN은 이미지 생성과정에 명령어를 실행시켜야 할 때 사용한다.
4 RUN apt update && apt install -y git
5
6 # RUN은 ENTRYPOINT 와 다르다.(생성된 이미지를 기반으로 컨테이너를 생성한 후에 실행시킬 명령어)
7 ENTRYPOINT [ "/bin/bash" , "-c" , "sleep 500" ]

```

9) **WORKDIR** 명령어

```

Dockerfile X
Dockerfile > ...
1 FROM ubuntu
2
3 # 작업디렉토리 지정
4 WORKDIR /test
5
6 COPY ./ ./
7 ENTRYPOINT [ "/bin/bash" , "-c" , "sleep 500" ]

```

10) 서블릿 프로젝트 Dockerfile 로 이미지 만들기

```

Dockerfile X
Dockerfile > ...
1 # 1. Tomcat 10 이미지를 기본 베이스로 사용
2 FROM tomcat:10 (last pushed 3 days ago)
3
4 # 2. 기존 ROOT 애플리케이션 제거 (기본 welcome page 제거)
5 RUN rm -rf /usr/local/tomcat/webapps/ROOT
6
7 # 3. 프로젝트의 ROOT.war를 Tomcat webapps 폴더로 복사
8 COPY ROOT.war /usr/local/tomcat/webapps/ROOT.war
9

```

실습하기 - Dockerfile 스크립트를 이용한 이미지 생성 httpd)-ex02_Dockerfile_httpd 폴더참조

index.html 문서 준비

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<style>
  h1 {color : blue;}
</style>
</head>
<body>
<h1>Heejung 의 도커 실습</h1><hr><h2>Dockerfile 로 생성한 도커이미지
테스트</h2>
</body>
</html>
```

Dockerfile 스크립트

```
FROM httpd
COPY ./index.html /usr/local/apache2/htdocs
```

cmd 창에서

```
docker build -t my_http01 .
```

2>docker build -t my_http01 .

현재 폴더에 Dockerfile을 실행한다.

생성된 이미지 확인하기

```
docker images
```

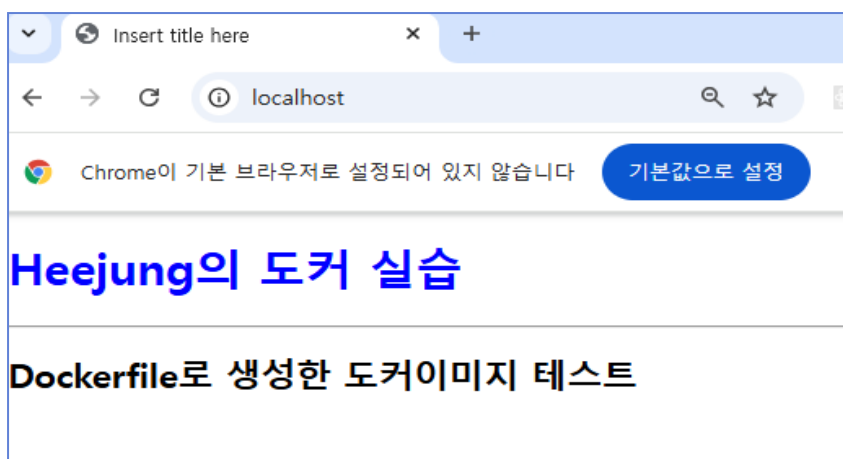
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my_http01	latest	acbf66f9b937	29 seconds ago	221MB

생성된 이미지로 컨테이너 실행하기

```
docker run --name http_con01 -d -p 80:80 my_http01
```

```
D:\W>cd D:\wdocker_test
D:\W>docker build -t my_http01 .
[+] Building 1.0s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 94B
=> [internal] load metadata for docker.io/library/httpd:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 323B
=> [1/2] FROM docker.io/library/httpd:latest@sha256:391a8eb0c1ed464163da46099606a5ec293705118f3054d6c60f5957e248
=> => resolve docker.io/library/httpd:latest@sha256:391a8eb0c1ed464163da46099606a5ec293705118f3054d6c60f5957e248
=> CACHED [2/2] COPY ./index.html /usr/local/apache2/htdocs
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:2f10ccc4d417decde8c6a318f6899c779bc4ccd8f1aae79b559a60a5387ed53e
=> => exporting config sha256:abfbc25434fb323f20349688365c7a2d65bdf5f94b900d7b016ca33465fe8d66
=> => exporting attestation manifest sha256:cebd43f8a104caf3ec0d6b2fab4bce0fda32409178cd37696c9d64b3fbfe26d5
=> => exporting manifest list sha256:80171a5b7a2c546d65a93b5f31ec36689426ad8b070154d2e6a4cb0ed277c19f
=> => naming to docker.io/library/my_http01:latest
=> => unpacking to docker.io/library/my_http01:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/yxmla7u2oss2mzgxm5v5getbj
[+] Building 2.0s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 94B
```

```
D:\W>docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
my_http01 latest 22d26c10343f 3 minutes ago 221MB
mysql 8.4 0f775a92980b 2 months ago 1.05GB
D:\W>docker run --name http_con01 -d -p 80:80 my_http01
ffbae55c72ef7e049b2235ba4980ef19df0bad/6f020951f30821b72da58d984
D:\W>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ffbae55c72ef my_http01 "httpd-foreground" 5 seconds ago Up 3 seconds 0.0.0.0:80->80/tcp http_c
D:\W>
```



실습하기- nginx 를 나만의 이미지로 만들기(nginx)-ex03_Dockerfile_nginx 폴더참고

index.html 문서 준비

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<style>
  h1 {color : blue;}
</style>
</head>
<body>
<h1>Heejung 의 도커 실습 -nginx </h1><hr>
<h2>Dockerfile 로 생성한 도커이미지 테스트</h2>
<imgsrc= "images/docker.png" width= "200">
</body>
</html>
```

Dockerfile 스크립트

```
FROM nginx
WORKDIR /usr/share/nginx/html
RUN mkdir images
COPY docker.png images/docker.png
COPY index.html .
EXPOSE 80
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

https://hub.docker.com/_/nginx

[https://github.com/nginx/docker-](https://github.com/nginx/docker-nginx/blob/eaf8875a1967d24cea6ed8b37109075e39ed9e43/mainline/debian/Dockerfile)

[nginx/blob/eaf8875a1967d24cea6ed8b37109075e39ed9e43/mainline/debian/Dockerfile](https://github.com/nginx/docker-nginx/blob/eaf8875a1967d24cea6ed8b37109075e39ed9e43/mainline/debian/Dockerfile)

구성 파일로 새 이미지를 빌드하세요

```
FROM nginx
COPY nginx.conf /etc/nginx/nginx.conf
```

Dockerfile에 사용자 정의를 추가하는 경우 nginx가 포그라운드에서 유지되도록 in을 CMD 포함해야 Docker가 프로세스를 제대로 추적할 수 있습니다(그렇지 않으면 컨테이너가 시작 직후에 즉시 중지됩니다)! -g daemon off; CMD

cmd 창에서

```
docker build -t my_nginx .
```

생성된 이미지 확인하기

```
docker images
```

생성된 이미지로 컨테이너 실행하기

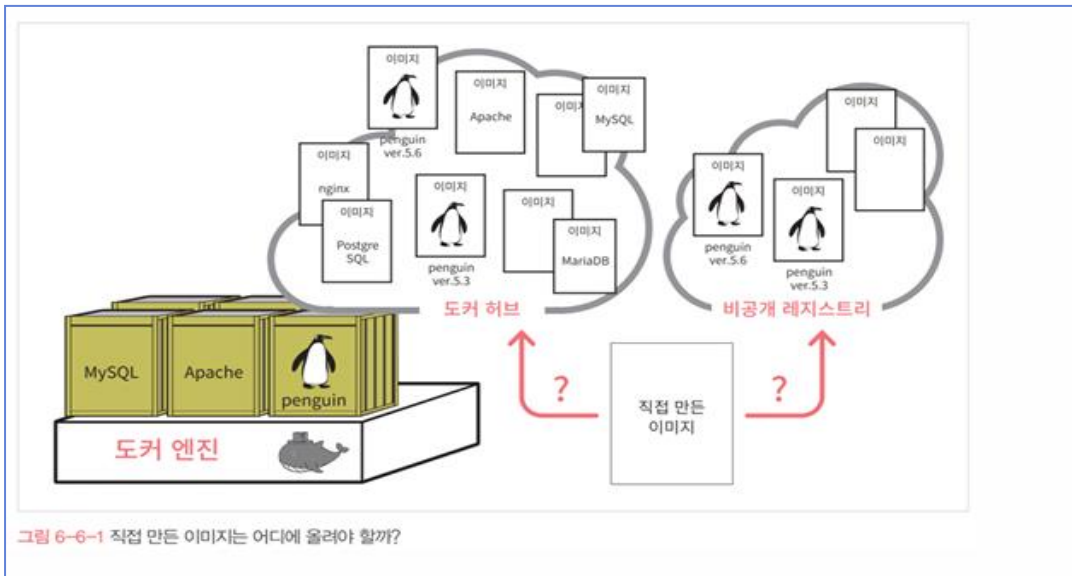
```
docker run --name nginx01 -d -p 80:80 my_nginx
```

실행결과



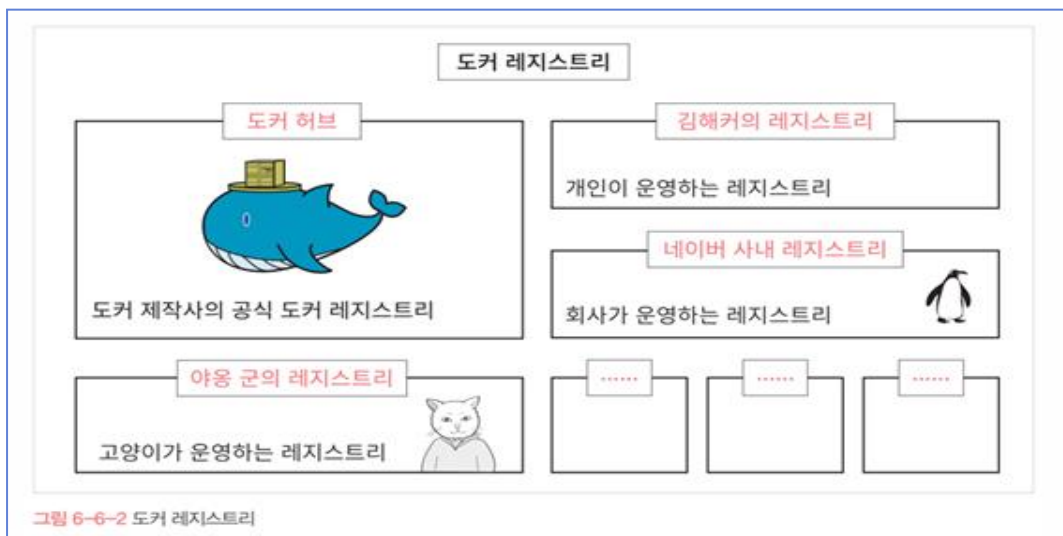
도커허브 - 도커이미지 레지스트리

직접 만든 이미지를 도커 허브에 업로드 할 수 있고, 비공개로 도커 허브와 같은 장소(레지스트리)를 만들 수도 있다.



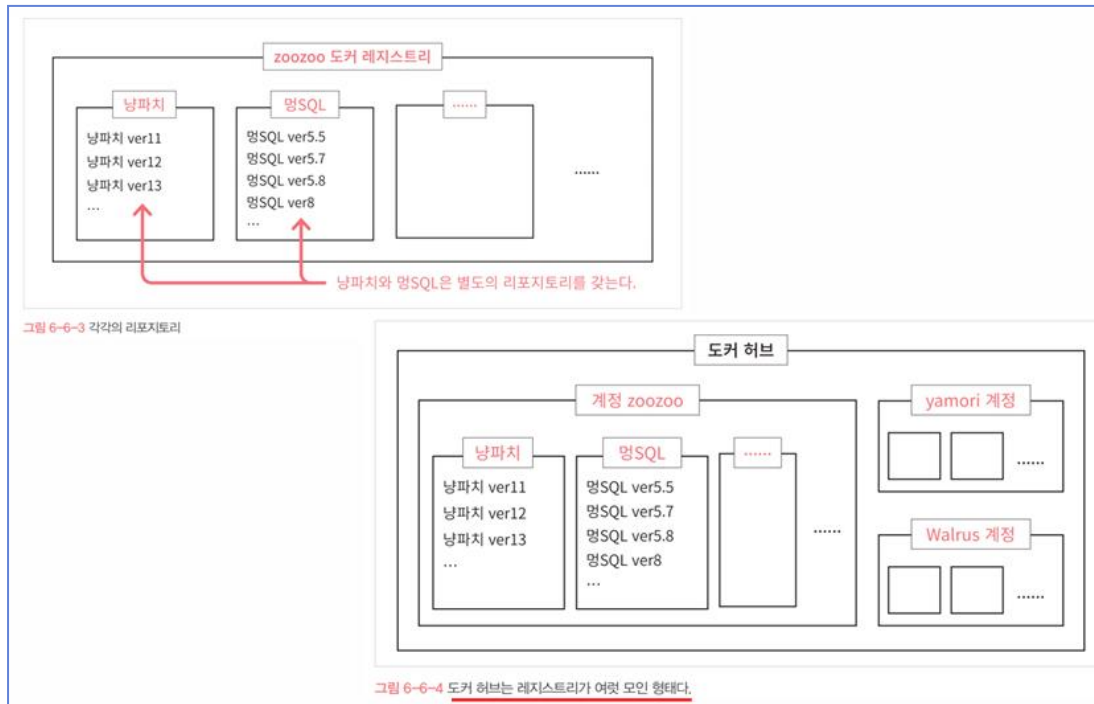
도커레지스트리 : 이미지 배포하는 장소

도커허브 : 도커 제작사에서 운영하는 공식 도커 레지스트리



레지스터리(Registry) : 이미지를 배포하는 장소

리포지터리(Repository) : 레지스트리를 구성하는 단위



태그 : 레지스트리 주소와 버전 표기

<형식>

‘레지스트리_주소(도커허브는 ID)/리포지터리_이름:버전’

버전은 생략 가능

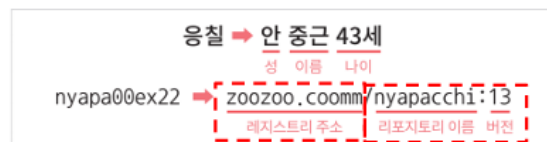


그림 6-6-5 태그를 붙이는 방법

이미지에 태그를 부여하여 복제하는 명령

`docker tag 원본_이미지_이름 레지스트리_주소/리포지터리_이름:버전`

이미지를 업로드하는 명령

`docker push 레지스트리_주소/리포지터리_이름:버전`

나만의 이미지를 push 하기

```
# 이미지 검색
docker images
```

이미지에 태그를 부여하여 복제

docker tag my_nginx jangheejung/my_nginx - tag 명시하지 않으면 latest

docker tag my_nginx jangheejung/my_nginx:v01 - v01 태그명시

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my_nginx	latest	6e19c003f55e	32 minutes ago	279MB
jangheejung/my_nginx	latest	6e19c003f55e	32 minutes ago	279MB

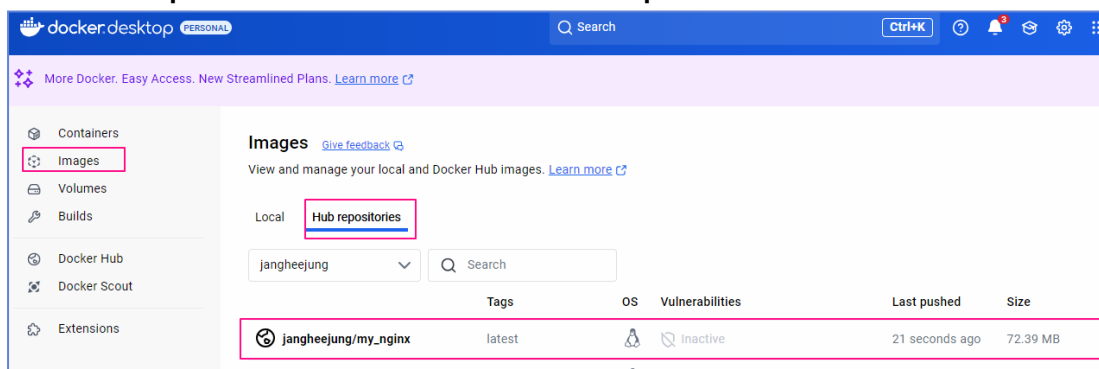
#이미지검색

docker images

#docker hub 에 push

docker push jangheejung/my_nginx

dockerDesktop 이나 **hub.docker.com** 에 접속해서 push 된 이미지를 확인한다.



local 에 있는 **jangheejung/my_nginx** 이미지를 삭제한다.

docker image rm jangheejung/my_nginx

push 한 이미지 pull + create + start

docker run --name nginx_v0_hub -d -p 80:80 jangheejung/my_nginx

#브라우저로 실행해보기

http://localhost/

← → ↻ localhost

 Chrome이 기본 브라우저로 설정되어 있지 않습니다 [기본값으로 설정](#)

Heejung의 도커 실습 -nginx

Dockerfile로 생성한 도커이미지 테스트



짝공이 올린 이미지를 pull 하여 실행 해보자